

Udhëzues Informativ mbi JavaScript

1. Çfarë është JavaScript?

JavaScript (JS) është një gjuhë programimi e nivelit të lartë, e interpretuar dhe me tipizim dinamik. Ajo përdoret kryesisht për të krijuar përmbajtje interaktive dhe dinamike në ueb-faqe. Së bashku me HTML dhe CSS, JavaScript është një nga tri teknologjitë bazë të uebit.

2. Historia dhe Origjina

JavaScript u krijua në shtator të vitit 1995 nga Brendan Eich gjatë kohës që punonte në kompaninë Netscape. Emri i saj i parë ishte Mocha, më pas u quajt LiveScript, dhe përfundimisht mori emrin JavaScript. Standardi zyrtar i gjuhës mirëmbahet nga organizata ECMA International nën emrin ECMAScript. Versioni ECMAScript 6 (ES6), i publikuar në vitin 2015, solli ndryshime rrënjësore në gjuhë.

3. Karakteristikat Kryesore

- Single-threaded dhe Asinkrone:** Ekzekuton kodin në një thread të vetëm por përdor mekanizmin Event Loop për të menaxhuar operacionet asinkrone pa bllokuar ekzekutimin.
- Tipizim Dinamik:** Lloji i të dhënave të një variabël përcaktohet gjatë kohës së ekzekutimit (runtime) dhe mund të ndryshojë.
- Trashëgimi bazuar në Prototipe:** Objektet trashëgojnë vetitë dhe metodat drejtpërdrejt nga objekte të tjera (prototipe).
- Multi-paradigmë:** Mbështet programimin e orientuar nga objektet, programimin funksional dhe programimin procedural.

4. Sintaksa Bazë dhe Deklarimi i Variablave

Në JavaScript modern, variablat deklarohen duke përdorur tri fjalë kyçe:

- const:** Përdoret për vlera që nuk ndryshojnë pas caktimit fillestar. Has fushëveprim blloku (block scope).
- let:** Përdoret për variabla që mund të marrin vlera të reja. Has fushëveprim blloku (block scope).
- var:** Mënyra e vjetër e deklarimit. Has fushëveprim funksioni (function scope) dhe duhet shmangur në kodin modern.

```
const emri = "JavaScript"; let akses = true; function pershendetje(perdorues) {  
  return "Përshëndetje " + perdorues + " në " + emri; }  
}
```

5. Ekosistemi dhe Përdorimi

JavaScript përdoret në mjedise të ndryshme zhvillimi:

- **Frontend (Client-side):** Përdoret brenda shfletuesit për interaktivitet. Kornizat kryesore përfshijnë React, Vue.js, Angular dhe Svelte.
- **Backend (Server-side):** Përdorimin e mundësoi mjedisi Node.js dhe më vonë Deno e Bun. Kornizat kryesore përfshijnë Express.js dhe NestJS.
- **Aplikacione Celulare:** Përdoren korniza si React Native dhe NativeScript për të ndërtuar aplikacione iOS dhe Android.
- **Aplikacione Desktop:** Përdoren mjedise si Electron apo Tauri (p.sh. aplikacione si VS Code apo Discord).

6. Programimi Asinkron

Për të menaxhuar kërkesat në rrjet apo leximin e skedarëve pa ndërprerë ndërfaqen e përdoruesit, JavaScript përdor strukturat asinkrone:

- **Callbacks:** Funksione që kalojnë si argumente për t'u ekzekutuar më vonë.
- **Promises:** Objekte që përfaqësojnë përfundimin ose dështimin e një operacioni asinkron.
- **Async / Await:** Sintaksë moderne mbi Promises që e bën kodin asinkron të duket si kod sekuencial.

```
async function merrTeDhenat(url) { try { const pergjigja = await fetch(url); const data = await pergjigja.json(); return data; } catch (gabim) { console.error("Gabim gjatë kërkesës:", gabim); } }
```

7. Koncepte të Tjera me Rëndësi

- **DOM (Document Object Model):** Struktura me pemë e faqes HTML me të cilën JavaScript ndërvepron për të ndryshuar elementet ose stilet.
- **Closure:** Aftësia e një funksioni për të mbajtur qasje në variablat e fushëveprimit të tij prind, edhe pasi funksioni prind ka përfunduar.
- **TypeScript:** Një superset i JavaScript me tipizim statik që ndihmon në kapjen e gabimeve gjatë kohës së zhvillimit para ekzekutimit.

